

QueueFlow

System Design, Architecture, and Engineering Rationale

A code-grounded analysis of a durable, tenant-aware distributed job execution platform built with PostgreSQL, AWS SQS, Python workers, Amazon S3, TypeScript/Express, React, Docker, Kubernetes, and Terraform.

PREPARED FROM THE WORKSPACE SOURCE TREE

Repository state analyzed: 27 August 2026

Scope: implementation, failure semantics, operations, validation, and roadmap

Abstract

QueueFlow is a portfolio-scale distributed job execution platform designed to demonstrate how asynchronous work can remain durable and understandable across database, queue, worker, and object-storage boundaries. A TypeScript/Express API accepts tenant-scoped requests; PostgreSQL holds authoritative job state, lifecycle events, worker registrations, and a transactional outbox; AWS SQS Standard provides at-least-once delivery and a dead-letter path; independent Python workers claim and execute jobs; Amazon S3 optionally stores result objects; and a React dashboard exposes the operational model. This paper reconstructs the system from its source code and deployment artifacts, explains why each major decision was made, analyzes failure windows and invariants, reports verification results, and identifies the boundary between the implemented educational system and a production scheduler. [R1-R17]

The central finding is that QueueFlow makes an appropriate and coherent trade: it does not attempt exactly-once transport. Instead, it combines durable intent, duplicate-tolerant delivery, atomic conditional claims, and guarded terminal transitions. This produces a small system whose reliability argument can be inspected directly. The largest remaining risks are operational and security-related rather than conceptual: plaintext API keys, broad unauthenticated metrics, a single heterogeneous queue, approximate priority, embedded control loops, limited integration testing, and incomplete production identity, network, and database infrastructure.

Keywords

distributed systems; job queue; transactional outbox; at-least-once delivery; idempotency; worker lease; PostgreSQL; SQS; multi-tenancy; resource-aware scheduling

Document map

Part	Purpose
1-3	Problem framing, requirements, and system context
4-7	Data, submission, dispatch, worker execution, recovery, and state-machine design
8-11	API, dashboard, deployment, observability, security, and testing
12-14	Tradeoffs, risks, evolution path, and conclusions
Appendices	Interfaces, configuration, invariants, and source inventory

Method note

This is a repository study, not a benchmark or production-readiness certification. Statements labeled as implemented are grounded in the current files. Rationale is inferred from code structure, comments, configuration defaults, README explanations, and failure-handling behavior. Verification was run locally where dependencies were available; AWS behavior was not exercised against live services.

1 Problem definition and design goals

QueueFlow addresses a deceptively difficult problem: accept a job quickly, preserve it through partial failures, execute it on an eligible worker, expose a coherent lifecycle to a tenant, and retain the result. The repository deliberately chooses a small workload catalog - text analysis, simulated compute, and intentional failure - so the engineering focus stays on orchestration rather than application logic. GPU is modeled as a capacity integer but no real device is allocated. [R1, R9]

Functional goals

- Accept and validate asynchronous job submissions through REST, the dashboard, or the Python SDK.
- Enforce per-tenant active-job quotas and tenant isolation on job and event reads.
- Persist a durable lifecycle: queued, running, retrying, completed, or failed, with append-only events.
- Dispatch only when at least one recently healthy worker advertises sufficient CPU, memory, and modeled GPU capacity.
- Handle duplicate messages, workload errors, malformed payloads, worker crashes, long executions, and API replica concurrency.
- Support local development, container deployment, Kubernetes placement, AWS queue/object infrastructure, and automated checks.

Non-goals

- Exactly-once execution or automatic idempotency of arbitrary external side effects.
- Strict priority ordering, targeted placement, real GPU allocation, preemption, gang scheduling, or HPC topology awareness.
- A complete production control plane with tenant administration, credential rotation, cancellation, autoscaling, tracing, or managed PostgreSQL.

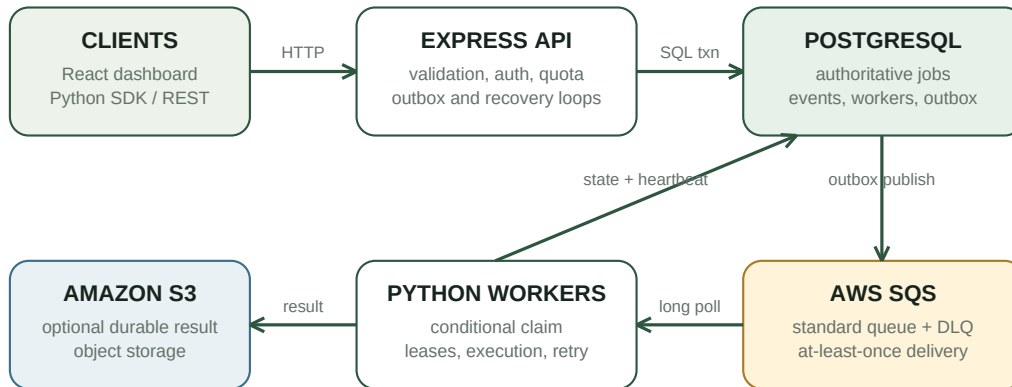
Design thesis

The system treats a queue message as permission to attempt a database claim, not as permission to execute. PostgreSQL is the serialization point for business truth; SQS is a lossy, duplicating, independently available delivery mechanism. This separation is the basis of QueueFlow's correctness argument.

Quality attributes and priorities

Attribute	Implemented mechanism	Consequence
Durability	job, SUBMITTED event, and outbox row in one transaction	Accepted work survives API or SQS outages
Consistency	conditional SQL updates and guarded terminal writes	Only one live claim should execute; stale owners cannot commit
Availability	API returns after database commit; publisher retries independently	SQS downtime does not make submission non-durable
Scalability	stateless HTTP replicas, independent workers, SKIP LOCKED	API and worker pools scale separately at modest load
Fairness	priority plus bounded age boost	Old work eventually competes, without strict ordering
Operability	JSON logs, health, Prometheus text, lifecycle events	Failures are observable, though metrics remain basic

2 System context and component architecture



CONTROL PRINCIPLE

PostgreSQL decides whether work may run. SQS transports a hint that work is available.

Duplicate delivery is expected; conditional state transitions make duplicate execution unlikely and stale commits invalid.

Figure 1. Logical architecture. The database is the authority; arrows show the principal data and control paths.

The API is both the synchronous edge and a lightweight control-plane host. It authenticates and validates HTTP requests, performs transactional submission, exposes reads, and starts two background loops: an outbox publisher and a stale-worker recovery scanner. Multiple replicas may run these loops because database row locks coordinate them. Workers are data-plane executors: each long-polls one SQS message at a time, atomically claims the corresponding job, renews visibility and heartbeat leases, executes outside any database transaction, persists the result, and commits a guarded terminal transition. [R2-R8]

Why this decomposition?

Choice	Reason	Tradeoff
PostgreSQL authority	ACID transactions can atomically join quota, state, event, and outbox decisions	Database becomes a critical shared dependency
SQS Standard transport	Managed long polling, visibility, retry, and DLQ semantics; duplicates are acceptable by design	No strict ordering or worker targeting
Python workers	Simple AWS/database integration and natural fit for future compute/data workloads	Two-language stack increases packaging and shared-contract burden
S3 result objects	Separates potentially durable artifacts from the operational database; versioning protects overwrites	Upload and completion cannot share one atomic transaction
React/Vite dashboard	Fast, small operational UI with demo and live modes	Some displayed values are illustrative, not derived telemetry
Embedded publisher/recovery	Keeps deployment topology small and makes replica safety demonstrable	Control duties scale with API replicas and complicate metrics

Technology selections

Express 5 and Zod provide a thin TypeScript HTTP boundary; node-postgres exposes explicit SQL and transaction control; AWS SDK v3 isolates SQS publication; psycopg2 gives the worker direct transactional access; boto3 covers SQS and S3; React 19, Vite, and Lucide build the dashboard; PostgreSQL 16 is used locally and in CI; Terraform constrains AWS provider versions; and Docker/Kubernetes encode repeatable process boundaries. [R2, R9-R16]

3 Data model and state ownership

The schema comprises five domain tables plus a migration ledger. UUIDs identify tenants, jobs, workers, and outbox records; job events use an ordered bigserial key. PostgreSQL CHECK constraints reinforce resource and priority bounds even though the API validates them earlier. Indexes support tenant/status lookup, dispatch ordering, event timelines, heartbeat scans, and pending outbox scans. [R7]

Table	Responsibility	Important constraints / indexes
tenants	identity, plaintext API key, active-job ceiling	unique api_key; max_active_jobs > 0
jobs	authoritative state, requirements, assignment, progress, result/error	tenant/status and dispatch indexes; bounded progress and priority
workers	capacity advertisement, current assignment, liveness	heartbeat index; positive CPU/memory; nonnegative GPU
job_events	append-only lifecycle and diagnostic narrative	cascade on job deletion; job/created_at index
job_outbox	durable publication intent, lock ownership, retry schedule	one row per job; partial pending index
schema_migrations	applied SQL filenames	primary key prevents duplicate application

State ownership rules

- The jobs row is canonical for current state; events explain transitions but are not replayed to rebuild state.
- SQS is not queried to determine job status. Its receive count influences retry delay and redrive only.
- Worker rows are soft-state capacity and liveness advertisements stored durably for coordination.
- The outbox persists the intent to publish exactly one logical job message; publication may physically occur more than once.
- Results may exist in both jobs.result JSONB and S3; result_location links the durable object when configured.

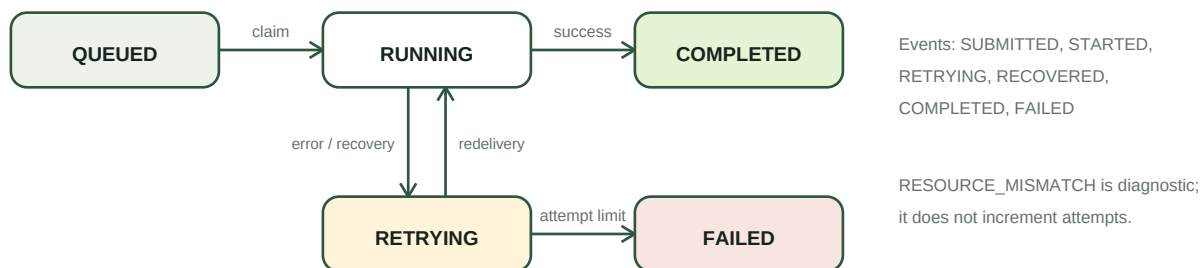


Figure 2. Job status state machine. Database updates and lifecycle events are paired in short transactions.

Invariant inventory

Invariant	Enforcement
An accepted job has durable publication intent	single submission transaction
Only QUEUED or RETRYING jobs can be claimed	conditional UPDATE predicate
A claim increments attempts exactly when acquired	same UPDATE statement
An owner can commit only while RUNNING and still assigned	worker_id and status guards
Tenant reads cannot cross tenant boundaries	tenant_id predicates on jobs and event join
Outbox rows are claimed by at most one live publisher at a time	FOR UPDATE SKIP LOCKED plus lock owner

4 Submission, quotas, and the transactional outbox

Submission is separated into a synchronous durability decision and asynchronous delivery. The request path ends after PostgreSQL commits; queue availability affects dispatch latency, not whether the accepted job exists. This separation is implemented by the transactional outbox.

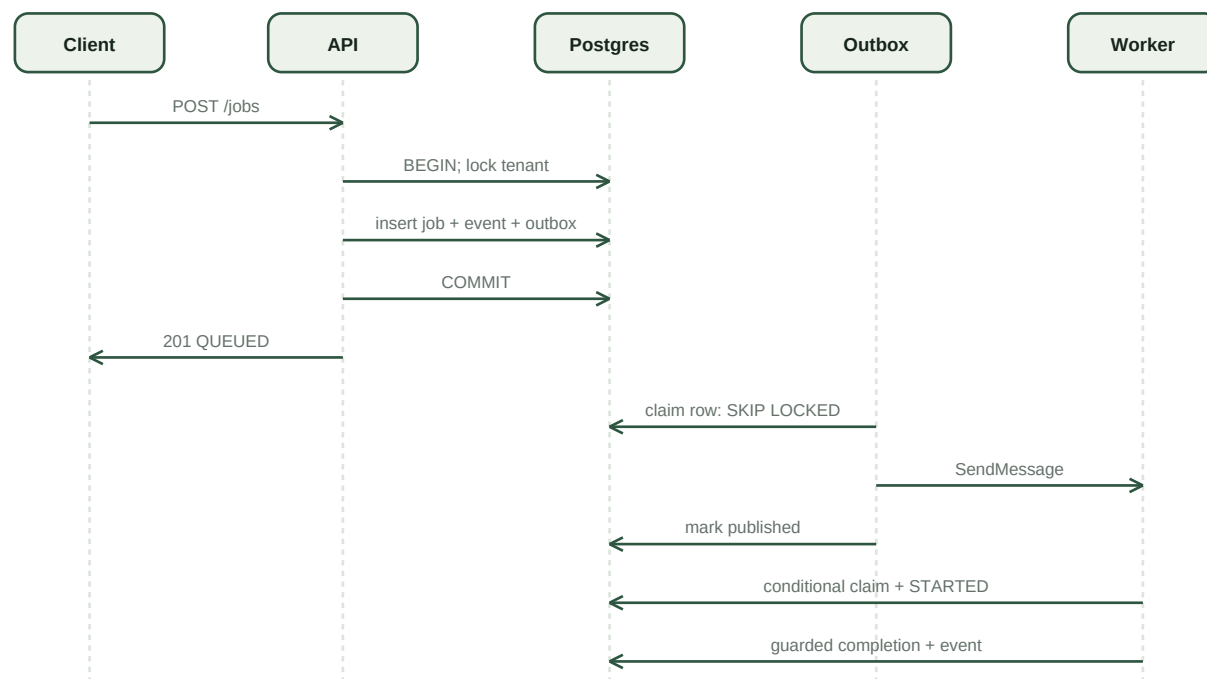


Figure 3. Submission-to-completion sequence. Work execution and S3 upload deliberately occur outside database transactions.

POST /jobs first parses a strict Zod schema. Unknown keys, malformed JSON, unsupported types, missing text for text analysis, non-integer or out-of-range resources, and invalid priority return HTTP 400. The Express JSON body limit is 128 KiB while text is capped at 100,000 characters, creating aligned protection against unbounded request memory. [R2, R8]

Quota serialization

Inside a database transaction, the service locks the tenant row with FOR UPDATE before counting active jobs. This intentionally serializes concurrent submissions for the same tenant, preventing two requests from both observing spare quota. It does not serialize tenants against each other. After the check, the service inserts the QUEUED job, a SUBMITTED event, and the outbox payload, then commits. A quota failure rolls the entire transaction back and maps to HTTP 429. [R3]

Why not send directly to SQS?

A direct database-then-SQS workflow has two fatal windows: committing the job and crashing before publish loses delivery; publishing first and failing the database commit creates an orphan message. The transactional outbox resolves this by committing database state and publication intent together. A background publisher then bridges the transactional boundary. This yields at-least-once publication: if the process crashes after SQS accepts the message but before `published_at` is stored, the row can be sent again after lock expiry. Duplicate-safe workers are therefore mandatory, not optional.

Publisher coordination and retry

- Every publisher has a UUID and claims one eligible row in a short transaction.
- FOR UPDATE OF o SKIP LOCKED lets API replicas make progress without blocking on the same row.
- A timed lock expiry recovers claims abandoned by a crashed publisher.
- Failed sends clear ownership, store a truncated error, and schedule bounded exponential backoff with 0.8-1.2 jitter.
- Successful sends set `published_at` only if the row is still owned by that publisher.

Dispatch eligibility and aging

The candidate query requires the job to remain QUEUED and requires at least one recently healthy, non-offline worker whose capacity satisfies the job. Eligible rows are ordered by priority plus an age boost: $\text{priority} + \min(10, \text{floor}(\text{age_seconds} / \text{PRIORITY_AGING_SECONDS}))$. With the default 60 seconds, a waiting job gains one effective point per minute, capped at ten. This is a pragmatic anti-starvation policy, but it is only dispatch priority. SQS Standard can reorder messages after publication. [R4]

5 Worker protocol and execution semantics

Each worker registers a stable configured ID or a generated startup UUID, advertises capacity, and heartbeats IDLE before each 20-second SQS long poll. One message is requested at a time, so the process has a straightforward single-job concurrency model. On shutdown, the worker stops polling, immediately releases any message received after shutdown was requested, finishes the current handler when grace permits, and finally marks itself OFFLINE. [R9]

Atomic claim

Receiving a message does not authorize execution. `claim_job` performs a single UPDATE that requires status in {QUEUED, RETRYING} and resource requirements within this worker's capacity. The same statement sets RUNNING, preserves the first `started_at`, increments `attempt_count`, assigns `worker_id`, and clears the prior error. If a row returns, STARTED and the BUSY heartbeat are committed in the same transaction. Concurrent deliveries race on one row; only the transaction that changes it receives the claim.

Non-claim outcomes

Outcome	Worker action	Rationale
MISSING	acknowledge	no authoritative work exists
ALREADY_COMPLETED	acknowledge	duplicate delivery is harmless
ALREADY_FAILED	retain; redrive eventually	preserves terminal failure evidence in DLQ
IN_PROGRESS	retain with delay	does not steal an active claim
RESOURCE_MISMATCH	record rate-limited diagnostic; retain longer	mismatch is not an execution attempt
NOT_EXECUTABLE	retain with retry delay	unknown/transient state is treated conservatively

Execution and result commit

Work runs outside the database transaction. Text analysis computes counts in memory; simulated compute sleeps and writes milestone progress; `always_fail` deliberately raises. The result is optionally written to S3 at `jobs/{job_id}/result.json`, then the worker conditionally marks the job COMPLETED only if it remains RUNNING and assigned to that worker. The completion event and worker return to IDLE share that transaction. If recovery removed ownership during a partition, the stale completion is rejected.

Failure and retry

On workload exception, the same ownership guard transitions the job to RETRYING or FAILED depending on `attempt_count` versus `max_attempts`, appends an event, and idles the worker. The SQS message is retained, and visibility is changed to bounded exponential backoff with jitter. Terraform configures `maxReceiveCount = 3`, matching the database default `max_attempts = 3`. These independent limits must be changed together to avoid surprising terminal behavior. [R9, R15]

Exactly-once boundary

QueueFlow provides duplicate-tolerant orchestration, not exactly-once effects. A claim can be recovered after a partition while an old worker continues an irreversible external action. The stale worker cannot commit QueueFlow state, but the external action itself must carry its own idempotency key or transactional protection.

6 Leases, crash recovery, and failure analysis

Visibility and heartbeat form two related leases. SQS visibility protects a received message from ordinary redelivery; the database heartbeat lets the control plane infer worker liveness. While a handler runs, a daemon thread wakes every 30 seconds by default, extends visibility to 120 seconds, and refreshes the BUSY heartbeat. Renewal failures are logged but do not abort the workload because connectivity may recover. The thread is always stopped and joined in finally. [R9, R16]

Stale-worker recovery

Each API replica periodically scans workers whose heartbeat is older than 120 seconds. It locks candidates with SKIP LOCKED, marks them OFFLINE, changes their assigned RUNNING jobs to RETRYING, clears ownership, records RECOVERED events, and commits the scan transaction. It does not publish a new message: the original SQS receipt is expected to become visible after its lease expires. This avoids creating a second delivery source and keeps recovery coupled to the existing redrive path. [R5]

Failure point	Observed state	Recovery / safety mechanism	Residual risk
API before commit	no accepted job	transaction rollback	client must retry request
API after commit, before publish	pending outbox row	publisher retry	submission latency rises
Publisher after SQS accept, before mark	message plus pending outbox	lock expiry and duplicate publish	duplicate delivery
Worker after receive, before claim	job still executable	visibility expiry	extra latency
Worker after claim, before completion	RUNNING assignment	heartbeat timeout + SQS redelivery	false recovery during partition
S3 write before DB completion	object may exist without completed state	later deterministic overwrite; bucket versioning	orphan or multiple object versions
Stale worker completes after recovery	assignment has been cleared	guarded completion rejects write	external side effects may already exist
Malformed queue message	no job can be parsed	non-acknowledgement sends it to DLQ	DLQ operator workflow is not implemented

Timing tradeoff

The stale threshold should comfortably exceed both ordinary heartbeat intervals and visibility-renewal intervals. Lower thresholds improve recovery time but increase false positives during database partitions or pauses. Higher thresholds reduce duplicate work but extend time-to-retry. QueueFlow exposes the parameters rather than hard-coding a single operational answer.

7 Scheduling and priority design

QueueFlow implements admission-aware dispatch, not full scheduling. Before publication, it proves that at least one healthy worker currently has sufficient advertised capacity. At claim time, the receiving worker repeats the check atomically. This two-stage design prevents clearly impossible jobs from immediately cycling through SQS while preserving correctness if capacity changes between dispatch and receipt.

Why one queue?

One SQS queue minimizes infrastructure and makes redrive, monitoring, and worker logic easy to explain. It is suitable for a small, mostly homogeneous pool. The cost is placement blindness: SQS cannot direct a message to a particular compatible worker. An undersized worker can receive a job and temporarily hide it from a suitable worker, causing head-of-line effects and consuming receive attempts. QueueFlow mitigates this with a distinct resource-mismatch delay, but cannot eliminate the topology problem.

Priority semantics

Property	QueueFlow behavior	Not guaranteed
Input range	integer 0-10, validated in API and database	arbitrary weights or classes
Dispatcher order	priority plus bounded age boost, oldest first on ties	global execution order
Starvation control	old jobs can gain up to 10 effective points	formal tenant fairness
Queue behavior	SQS Standard after publication	FIFO or strict priority
Placement	existence check plus worker-side capacity guard	reservation or targeted assignment

Scale-out path

- Introduce resource-class queues (for example CPU-small, CPU-large, GPU) to reduce unsuitable receives.
- Move eligibility and assignment into a dedicated scheduler with reservations, fairness, and leadership.
- Represent allocatable versus total capacity and permit controlled concurrency per worker.
- Add tenant-weighted fair sharing, queue depth metrics, cancellation, deadlines, preemption, and autoscaling signals.
- For real accelerators, add node/device discovery, topology, isolation, artifact staging, and workload sandboxing.

Decision assessment

For the stated portfolio goal, the single-queue design is a good boundary: it demonstrates capability checks and honest limitations without disguising a queue as a cluster scheduler. For heterogeneous production compute, the same choice becomes the first architectural constraint to remove.

8 API, tenancy, SDK, and dashboard

The public surface is intentionally small. /health and /metrics are unauthenticated. All /jobs and /workers routes pass through x-api-key authentication. A successful key lookup attaches tenantId to the request; job list, detail, and event queries include that tenant ID. Worker inventory is cluster-wide even when accessed by a tenant. [R2]

Endpoint	Auth	Purpose
GET /health	none	database connectivity and current database time
GET /metrics	none	Prometheus text: process counters and database gauges
POST /jobs	API key	validate and transactionally submit a job
GET /jobs	API key	tenant-scoped jobs ordered by priority and recency
GET /jobs/:id	API key	tenant-scoped job detail
GET /jobs/:id/events	API key	tenant-scoped chronological lifecycle
GET /workers	API key	cluster-wide worker capacity and health

Authentication and tenancy rationale

API keys make the sample easy to exercise with curl, the SDK, and the browser. Tenant-scoped SQL predicates are the primary isolation boundary, and a tenant-row lock also becomes the quota serialization point. The model is adequate for demonstration but not production: keys are stored and compared in plaintext, there is no rotation, audit trail, per-route authorization, rate limiting, or tenant-scoped worker view. A production design should store only keyed hashes, show secrets once, rotate credentials, and enforce network and application-level rate policies.

Python SDK

QueueFlowClient is a deliberately thin wrapper around requests. It normalizes the base URL, attaches the API key, submits resource parameters, fetches jobs and events, and polls until a terminal status. This keeps the API transparent. Missing refinements include typed results, configurable timeouts and backoff, cancellation, async support, pagination, and retry policy. The SDK still recognizes SUBMISSION_FAILED although the current durable outbox design does not emit that status. [R10]

Dashboard design

The React interface provides overview, jobs, workers, submission, connection settings, and a job-detail timeline. Demo mode animates sample state without backend services; live mode stores the API URL/key in localStorage and polls jobs/workers every four seconds. A responsive, editorial visual system uses a dark forest palette, lime activity accents, compact status pills, metric cards, a table, modals, and a slide-over timeline. This makes distributed state legible to a portfolio reviewer. [R11-R12]

Dashboard caveats

- The displayed date and average runtime are hard-coded; success rate is derived client-side from loaded jobs.
- Live polling has no pagination, ETag, streaming, or cancellation and could become expensive with a large job table.
- Storing an API key in localStorage increases exposure to cross-site scripting; a production console should use a secure session.
- The UI submission form caps CPU at 8 and does not expose GPU, while the API accepts larger CPU and modeled GPU values.
- The nginx container proxies /api to the API service, allowing the browser to avoid direct cross-origin deployment complexity.

9 Deployment and infrastructure architecture

QueueFlow provides three deployment layers: Docker Compose for a local integrated stack, Kubernetes manifests for replicated API/worker processes, and Terraform for AWS queues and result storage. The layers intentionally leave PostgreSQL deployment and production networking to the operator. [R13-R16]

Docker Compose

Service	Role	Operational choice
postgres	PostgreSQL 16 on host port 5433	persistent named volume and readiness health check
api	built TypeScript service on 3001	waits for database; auto-migrates; mounts local AWS credentials read-only
web	nginx-served Vite build on 4173	depends on healthy API; /api reverse proxy
worker	Python executor	waits for API; 60-second stop grace; mounts local AWS credentials read-only

Kubernetes

The manifests run two API replicas and two worker replicas with resource requests/limits and a shared queueflow service account. The API has a readiness probe against /health; both deployments allow 60 seconds for termination. Configuration and secrets are imported from separate ConfigMap and Secret objects. Worker names derive from pod metadata. The repository correctly avoids mounting developer credential paths in Kubernetes and expects workload identity through the AWS SDK default chain.

Terraform

- A standard job queue uses 20-second receive wait, 120-second default visibility, four-day retention, 1 MiB messages, and SQS-managed encryption.
- A DLQ uses fourteen-day retention and an explicit byQueue redrive allow policy; maxReceiveCount is three.
- The S3 bucket blocks all public access, enables versioning, and uses AES-256 server-side encryption.
- Outputs expose the queue URLs and bucket name. The module does not create IAM policies, a database, networking, compute, alarms, lifecycle rules, or a remote Terraform backend.

Migration safety

The API runs sorted SQL migrations at startup under a PostgreSQL advisory lock. Each migration is transactional and recorded by filename. This permits concurrent API replicas to start safely. Demo tenant seeding is explicitly environment-controlled, enabled in Compose, and absent from Kubernetes defaults. In a larger organization, migrations should usually become a distinct release job to decouple schema change failure from every application replica. [R6]

10 Observability and operational model

Both API and worker logs are newline-delimited JSON on standard output. Fields include service, level, timestamp, job_id, worker_id, attempt, outbox ID, retry delay, status, and error. This is a sound container-native choice: log collection stays outside the process, while structured correlation remains possible. [R17, R9]

Health and metrics

GET /health executes SELECT NOW() and reports 503 if PostgreSQL is unavailable. It therefore measures the dependency required for all useful API operations, but does not test SQS, S3, publisher freshness, or recovery-loop health. GET /metrics combines in-memory API counters with database queries for job counts by status and active workers. The text follows the basic Prometheus exposition format.

Signal	Source	Scope / limitation
jobs_submitted	in-process counter	only submissions observed by one API replica; resets on restart
outbox_publish_failures	in-process counter	useful symptom, no durable aggregation
outbox_scan_failures	in-process counter	detects publisher DB/query failures
worker_recoveries	in-process counter	counts jobs recovered by one replica
queueflow_jobs{status}	PostgreSQL snapshot	durable cluster-wide current-state gauge
queueflow_active_workers	PostgreSQL snapshot	uses a fixed 90-second health window, distinct from configurable recovery

Operational gaps

- No histograms for submission latency, queue wait, runtime, result upload, retry delay, or recovery time.
- No gauge for pending/oldest outbox row, SQS depth, DLQ depth, lease-renewal failures, or capacity saturation.
- No distributed trace or correlation ID spanning HTTP, outbox, queue, worker, database, and S3.
- No liveness probe distinct from readiness, no worker health endpoint, and no alert definitions or runbooks.
- Metrics are unauthenticated; production deployment must restrict them by network policy or private routing.

Recommended service-level indicators

The next operational layer should track accepted-submission durability, p50/p95/p99 time from created_at to started_at, execution duration, completion ratio by type and attempt, age of oldest unpublished outbox row, stale worker count, resource-mismatch rate, queue/DLQ depth, and S3/DB completion divergence. These directly measure the architecture's promises and failure windows.

11 Security, privacy, and trust boundaries

The principal trust boundaries are browser/client to API, API/worker to PostgreSQL, control plane to SQS, worker to S3, and deployment platform to AWS credentials. QueueFlow includes important baseline controls - request validation, parameterized SQL, tenant predicates, S3 public-access blocking, encryption at rest, separate Kubernetes Secret references, and non-root credential assumptions in production - but it is not security-complete.

Area	Current posture	Priority improvement
API credentials	plaintext unique keys in database; browser stores key locally	hash with server-side pepper, rotate, scope, audit, secure session for UI
Authorization	tenant job reads; cluster-wide workers	explicit roles and tenant/resource policy
Abuse resistance	body/resource limits and active-job quota	rate limiting, total payload/result quotas, concurrency and cost controls
Network	CORS configurable; no checked-in ingress/network policy	TLS termination, private database, network policies, service-to-service auth
AWS access	SDK default chain; local read-only credential mount	least-privilege workload identity and separate API/worker roles
Data	S3 private, encrypted, versioned; result also in DB	retention/lifecycle policy, KMS option, sensitive-text classification
Workload isolation	fixed built-in functions in worker process	sandboxing, resource enforcement, artifact verification for user code
Supply chain	lockfiles, CI builds/lint/tests	image scanning, SBOM, signed images, pinned actions/provider hashes

Threat observations

- SQL injection risk is reduced through parameterized queries, but API-key timing and storage practices remain basic.
- CORS is permissive when CORS_ORIGIN is unset. CORS is not authorization, and the key remains the real boundary.
- The unauthenticated metrics endpoint may reveal workload volume and cluster capacity.
- Result objects use guess-resistant job UUID paths but must still rely on IAM; obscurity is not access control.
- A malicious or compromised worker has direct database credentials and can write broadly; production should narrow database roles or move state transitions behind authenticated control-plane APIs.

Security decision boundary

The current direct-database worker protocol is excellent for making atomic claims explicit, but it expands the database trust boundary to every worker. That trade is acceptable in a controlled demonstration cluster and deserves reconsideration before untrusted workloads or multi-team operations.

12 Verification, testing, and evidence quality

The repository contains focused unit tests for validation, authentication, tenant scoping, transactional job creation, outbox failure retry, stale-worker recovery, atomic claims, duplicate delivery, capacity mismatch, retry limits, guarded completion, and safe acknowledgement. GitHub Actions builds the API, runs PostgreSQL migrations and tests, then installs, lints, compiles, and tests the worker. AWS clients are mocked. [R18-R23]

Check run on 27 Aug 2026	Result	Interpretation
API: npm run build	PASS	TypeScript API compiled
API: npm test	PASS	five top-level test files; nested functional cases passed
PostgreSQL integration case	SKIPPED	TEST_DATABASE_URL was not supplied to the local command
Worker: .venv unittest	PASS	7 tests passed
Worker: Ruff	PASS	worker and tests passed lint checks
Web: clean npm install + production build	PASS	1,578 modules transformed; production bundle emitted
Terraform validate	PASS	configuration reported valid
Docker Compose config	PASS	configuration parsed quietly

Evidence limitations

These checks demonstrate code-level consistency, not live distributed-system behavior. The local session did not exercise real SQS visibility expiry, duplicate delivery, DLQ redrive, IAM, S3 versioning, PostgreSQL failover, Kubernetes termination, or multi-replica load. The repository's integration test can validate atomic persistence and quota enforcement when a PostgreSQL URL is supplied, but it was skipped here. The paper therefore treats AWS and cluster behavior as design claims grounded in code/configuration, not empirical production results.

Highest-value next tests

- A credentialed ephemeral AWS/LocalStack staging suite covering send, visibility renewal, duplicate delivery, redrive, and result storage.
- A concurrent publisher test with multiple API replicas and deliberate crash after SendMessage but before published_at.
- A partition/lease test in which a stale worker attempts completion after database recovery changes ownership.
- Property-based state-machine tests that reject illegal transitions and preserve event/state pairing.
- Load tests for tenant quota contention, outbox latency, one-queue mismatch behavior, and database pool saturation.
- Security tests for cross-tenant reads, CORS configuration, oversized bodies, key rotation, and least-privilege IAM.

13 Design tradeoffs and critical assessment

Decision	What it buys	What it costs	Assessment
Transactional outbox	durable acceptance across DB/queue boundary	duplicate publish window and polling latency	strong, essential choice
Conditional DB claim	duplicate tolerance and ownership serialization	database on worker hot path	strong at this scale
Guarded completion	stale workers cannot overwrite recovered state	external effects remain outside guard	strong but incomplete effect safety
Single SQS Standard queue	minimal topology and managed retry/DLQ	no strict priority or targeted placement	appropriate prototype boundary
Embedded control loops	few deployables	per-replica loops/counters and scaling complexity	reasonable now; split later
Direct worker DB access	simple atomic protocol	large trust boundary and credential distribution	revisit for hostile multi-tenancy
S3 before completion	results durable before terminal state	non-atomic orphan/overwrite window	acceptable with reconciliation
Polling dashboard	simple live view	stale data and O(n) repeated reads	adequate for small demonstrations
Startup migrations	easy local and replica-safe startup	schema rollout coupled to serving	move to release job at maturity

Architecture strengths

- Failure windows are acknowledged and handled with explicit state transitions rather than hidden behind an exactly-once claim.
- Important state and the event that explains it are committed together in short transactions.
- Concurrency safety is visible in SQL: tenant locks, conditional updates, ownership guards, and SKIP LOCKED.
- Configuration makes timing, retry, and aging policies inspectable and tunable.
- Local, Kubernetes, and AWS artifacts are consistent with the logical decomposition and avoid embedding production secrets.

Architecture weaknesses

- No formal job-status enum or foreign key from jobs.worker_id/current_job_id, leaving some consistency to application code.
- No cancellation, timeouts, maximum result size, retention workflow, pagination, or administrative tenant lifecycle.
- Worker registration with generated IDs leaves historical rows to be marked offline rather than reused after restart.
- Resource advertisement represents total capacity, not allocatable capacity; one-job worker concurrency sidesteps accounting rather than solving it.
- No dedicated reconciliation for pending outbox age, S3/DB divergence, terminal messages, or DLO replay.

Overall judgment

QueueFlow is architecturally coherent for its stated purpose. Its reliability mechanisms are more important than its workload catalog, and they compose correctly around a single authority. It should be presented as a durable job-orchestration reference implementation, not as a production scheduler or exactly-once system.

14 Evolution roadmap

A responsible roadmap should preserve the current invariants while progressively hardening boundaries.

Horizon	Changes	Why now
1. Correctness hardening	status enums/constraints; foreign keys where practical; reconciliation jobs; terminal/DLQ operator workflow; idempotency key on submission; cancellation model	closes known consistency and operator gaps without changing topology
2. Security and operations	hashed rotating credentials; roles; network policy; workload identity; managed PostgreSQL/backups; aggregated metrics; traces; alerts; runbooks	makes the existing architecture supportable and safer
3. Scale separation	dedicated publisher/recovery service; pagination; event streaming; worker concurrency accounting; autoscaling; partitioned event/outbox data	reduces API coupling and database/query pressure
4. Scheduling maturity	resource-class queues or direct assignments; reservations; tenant fairness; deadlines; preemption; real accelerator discovery	addresses heterogeneous placement and priority semantics
5. Workload platform	sandboxed execution, artifact staging, signed images, result schemas, workflow DAGs, retry policy per job	expands from demo functions to controlled user workloads

Recommended next architecture

The highest-leverage near-term change is not a new scheduler. It is an operational reconciliation/control service that owns outbox publishing, recovery, DLQ inspection, and consistency checks, with durable Prometheus metrics and traces. This keeps the existing PostgreSQL authority and worker claim invariant while making control-plane health independently deployable and observable. Only after workload heterogeneity creates measurable queue mismatch should the system introduce resource-class routing or direct reservations.

Preserve these invariants

- Acceptance and durable dispatch intent remain atomic.
- Every execution begins with an authoritative conditional claim.
- Every terminal database transition verifies current ownership.
- Retries are bounded and visible in both state and events.
- Tenant isolation is enforced in the authoritative query, not only in the UI.
- External effects use stable idempotency keys whenever they cannot share the state transaction.

15 Conclusion

QueueFlow demonstrates a sound small-system answer to distributed job execution. It accepts work durably by committing the job, audit event, and outbox intent together. It publishes asynchronously with row-level coordination and backoff. It tolerates SQS duplicates by requiring a conditional PostgreSQL claim. It protects long work with visibility and heartbeat renewal, recovers abandoned claims using stale-worker scans, and rejects stale terminal commits through ownership guards. It also makes scheduling limitations explicit: capability-aware admission and aging are useful, but a single unordered queue is not a heterogeneous scheduler.

The implementation's most important design choice is epistemic clarity: each component has a narrow source of truth. PostgreSQL answers what the job is and who owns it. SQS answers what may need attention and when it should be retried. Worker heartbeats provide failure suspicion, not proof. S3 stores a result object but does not decide completion. The event log explains lifecycle changes without replacing the current-state row. These boundaries make both strengths and failure modes understandable.

For a portfolio or educational system, the architecture is credible and unusually honest. For production, the next work is clear: harden identity and authorization, aggregate observability, operationalize DLQ and reconciliation, exercise real distributed failures, narrow worker privileges, and evolve scheduling only when workload demands justify the complexity. The existing transactional and ownership invariants should remain the foundation.

Final assessment: a well-scoped durable orchestration platform with strong failure semantics, suitable as a reference architecture and foundation for further hardening.

Appendix A - Interface and configuration reference

Submission schema

Field	Rule	Default / note
type	text_analysis simulated_compute always_fail	required
text	trimmed string, 1-100,000 chars	required only for text_analysis
cpu	integer 1-128	1
memoryMb	integer 1-1,048,576	256
gpu	integer 0-64	0; modeled capacity only
priority	integer 0-10	0

Key timing defaults

Parameter	Default	Role
OUTBOX_POLL_INTERVAL_MS	1,000 ms	publisher scan cadence
OUTBOX_LOCK_TIMEOUT_SECONDS	30 s	abandoned publisher claim recovery
OUTBOX_RETRY_BASE_MS / MAX	1,000 / 60,000 ms	publish failure backoff bounds
WORKER_STALE_AFTER_SECONDS	120 s	database liveness suspicion threshold
WORKER_RECOVERY_INTERVAL_MS	30,000 ms	recovery scan cadence
PRIORITY_AGING_SECONDS	60 s	one effective priority point per interval
SQS_VISIBILITY_TIMEOUT_SECONDS	120 s	initial and renewed queue lease
SQS_VISIBILITY_RENEWAL_SECONDS	30 s	lease/heartbeat refresh cadence
RETRY_BASE_DELAY_SECONDS / MAX	5 / 60 s	work failure visibility backoff
RESOURCE_MISMATCH_DELAY_SECONDS	30 s	extra mismatch deferral

Operational coupling rules

- Visibility renewal interval must remain comfortably below visibility timeout.
- Stale-worker threshold should exceed normal heartbeat and lease-renewal intervals.
- Database max_attempts and SQS maxReceiveCount should change together.
- Priority aging must be nonzero and tuned against expected queue wait, not execution duration.
- SQS and S3 identifiers belong in configuration; production credentials should come from workload identity.

Appendix B - Repository source inventory

- [R1] README.md. Declared scope, architecture, operating model, limitations, and run instructions.
- [R2] apps/api/src/app.ts. HTTP routes, authentication, tenancy, health, metrics, and errors.
- [R3] apps/api/src/job-service.ts. Quota locking and atomic job/event/outbox submission.
- [R4] apps/api/src/outbox.ts. Publisher claim, capability gating, aging, locking, and retry.
- [R5] apps/api/src/recovery.ts. Stale-worker scan and atomic recovery.
- [R6] apps/api/src/migrate.ts. Advisory-locked startup migrations and demo seeding.
- [R7] apps/api/migrations/001_initial.sql. Relational model, constraints, and indexes.
- [R8] apps/api/src/validation.ts. Strict job schema and resource bounds.
- [R9] workers/processor/worker.py. Registration, claims, leases, execution, result storage, retry, shutdown.
- [R10] sdk/python/queueflow.py. Python client interface and polling semantics.
- [R11] apps/web/src/main.tsx. Dashboard interaction, demo/live modes, submission, and timeline.
- [R12] apps/web/src/styles.css. Responsive visual design system.
- [R13] docker-compose.yml. Local service topology and configuration.
- [R14] k8s/queueflow.yaml and k8s/config.example.yaml. Replicated Kubernetes runtime and configuration boundaries.
- [R15] infra/terraform/main.tf. SQS, DLQ, encryption, redrive, and S3 controls.
- [R16] .env.example. Documented control-plane and worker timing defaults.
- [R17] apps/api/src/logger.ts and metrics.ts. Structured logs and process-local counters.
- [R18] apps/api/test/app.test.ts. Authentication, tenancy, and validation HTTP tests.
- [R19] apps/api/test/job-service.test.ts. Submission transaction and quota rollback tests.
- [R20] apps/api/test/outbox-recovery.test.ts. Publisher failure and recovery tests.
- [R21] apps/api/test/database.integration.test.ts. Optional PostgreSQL integration test.
- [R22] workers/processor/tests/test_worker.py. Claim, duplicate, mismatch, retry, and completion tests.
- [R23] .github/workflows/ci.yml. Build, migration, lint, and test automation.

Repository scope note

The workspace had pre-existing uncommitted changes at review time. This paper analyzes the working tree visible on 27 August 2026 and does not assume that every analyzed file is present in the latest Git commit. Dependency directories and generated build artifacts are excluded from architectural conclusions.

Appendix C - Compact correctness argument

Claim 1: accepted submissions are not silently lost between PostgreSQL and SQS.

The API commits the job and its unpublished outbox row in the same transaction. If the transaction aborts, the request is not accepted. If it commits, the row remains queryable until a publisher records `published_at`. Publication failures retain the row and set `next_attempt_at`. Therefore an SQS outage cannot erase accepted intent, assuming PostgreSQL durability and an eventually running publisher.

Claim 2: duplicate SQS messages do not normally cause concurrent execution of one job.

All workers use one conditional UPDATE on the same jobs row. Only a row currently QUEUED or RETRYING can return. PostgreSQL serializes concurrent updates to that row, so after one transaction changes status to RUNNING, another cannot satisfy the predicate. A duplicate for COMPLETED is acknowledged without execution; a duplicate for RUNNING is retained.

Claim 3: a recovered stale owner cannot overwrite authoritative terminal state.

Recovery changes RUNNING to RETRYING and clears `worker_id`. Completion and failure both require status RUNNING and `worker_id` equal to the committing worker. Once recovery commits, the stale worker's predicate matches no row and its state commit is rejected. This does not undo non-transactional external effects, which require workload-level idempotency.

Claim 4: lifecycle events are consistent with key state transitions.

SUBMITTED shares the submission transaction; STARTED shares the claim transaction; COMPLETED/RETRYING/FAILED share the terminal or retry transaction; RECOVERED shares the recovery transaction. Thus these events do not claim a transition that rolled back. Progress updates and RESOURCE_MISMATCH diagnostics are separate and intentionally less strict.

Assumptions

- PostgreSQL provides its documented ACID and row-lock semantics and remains durably configured.
- At least one publisher and eligible worker eventually run with valid credentials and connectivity.
- SQS eventually redelivers non-deleted messages within retention and applies the configured redrive policy.
- Operators keep timing and attempt limits coherent and provision sufficient capacity.
- User workload side effects are idempotent or otherwise protected when exactly-once effects matter.